

🌱 AI-Powered Smart Irrigation System

Predictive Water Management and Crop Optimization

An AI-powered smart irrigation system that combines **machine learning**, live soil-moisture sensor data, weather forecasts, crop information, and irrigation history to determine irrigation requirements and generate field-specific irrigation schedules.

📌 Overview

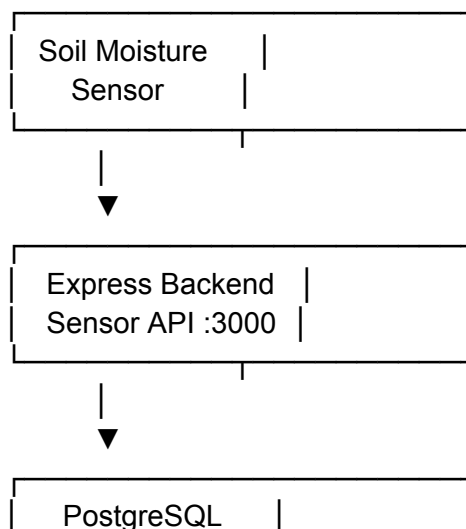
Traditional irrigation often relies on fixed schedules or manual decisions. This project uses an AI-based decision-support approach to dynamically determine:

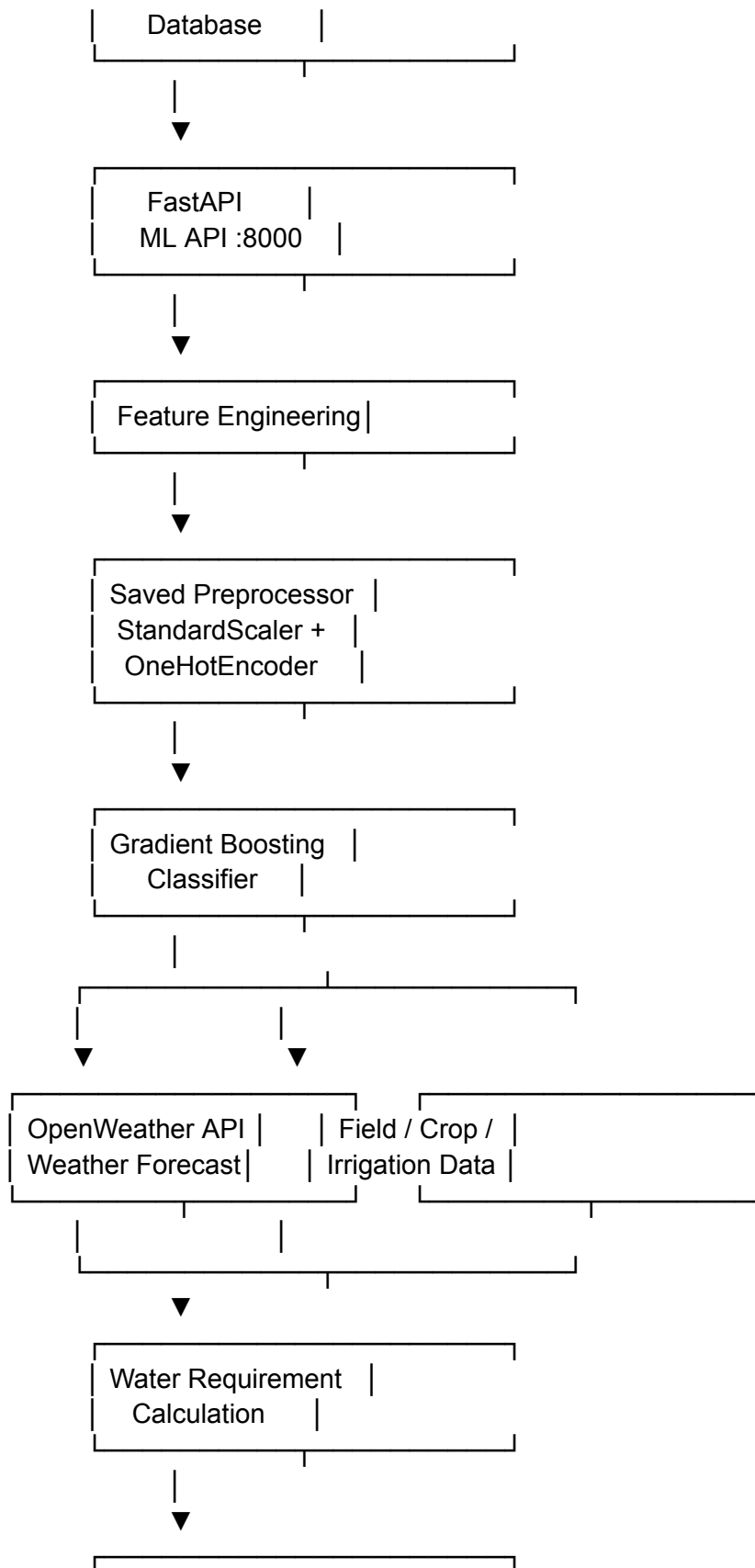
- * Whether irrigation is required
- * Irrigation requirement: **Low / Medium / High**
- * Approximate water requirement
- * Recommended irrigation time
- * Field-specific irrigation schedule
- * Over-watering prevention

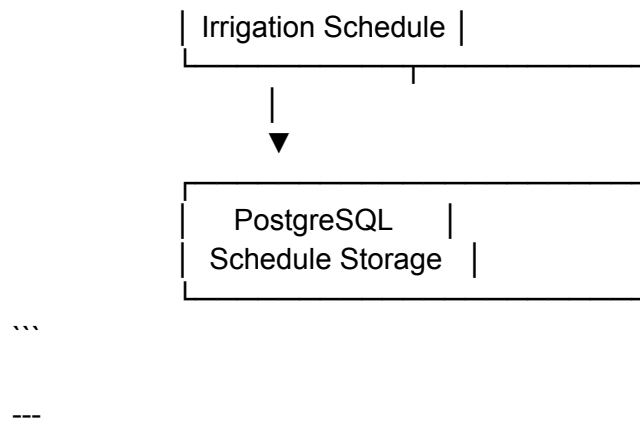
The system integrates real-time sensor readings with an ML prediction engine and weather forecasts.

🏗️ System Architecture

```text







## # 📁 Project Structure

```

``text
smart-irrigation-system/
├── backend/
│ ├── api/
│ │ ├── __init__.py
│ │ └── main.py
│ ├── server.js
│ └── .env
├── database/
│ └── schema.sql
├── ml/
│ ├── data/
│ │ └── irrigation_prediction.csv
│ ├── models/
│ │ ├── irrigation_gb_model.pkl
│ │ ├── irrigation_preprocessor.pkl
│ │ └── model_version.json
│ └── src/
│ ├── feature_engineering.py
│ ├── mlflow_tracking.py
│ ├── model_version.py
│ ├── predict.py
│ ├── prediction_engine.py
│ └── preprocessing.py

```

```

| | |— test_feature_engineering.py
| | |— test_preprocessing.py
| | |— train_model.py
| | |
| | |— mlflow.db
| |
| |— simulation/
| | |— sensor_readings.py
| |
| |— .gitignore
|
...

```

> **Note:** `.env`, virtual environments, `node_modules`, Python cache files, and other development-only files are excluded through `.gitignore`.

---

# 🧠 Machine Learning

## Dataset

The ML system uses an irrigation dataset containing **10,000 records** and **20 original columns**.

### Input Features

```

``text
Soil_Type
Soil_pH
Soil_Moisture
Organic_Carbon
Electrical_Conductivity
Temperature_C
Humidity
Rainfall_mm
Sunlight_Hours
Wind_Speed_kmh
Crop_Type
Crop_Growth_Stage
Season
Irrigation_Type
Water_Source
Field_Area_hectare
Mulching_Used

```

```
Previous_Irrigation_mm
Region
'''
```

```
Target
```

```
'''text
Irrigation_Need
'''
```

Possible target classes:

```
'''text
Low
Medium
High
'''
```

```

```

```
 Feature Engineering
```

Feature engineering is implemented in:

```
'''text
ml/src/feature_engineering.py
'''
```

The following features are generated.

| Feature                         | Logic                              |
|---------------------------------|------------------------------------|
| `Soil_Moisture_Deficit`         | `100 - Soil_Moisture`              |
| `Soil_Moisture_Stress`          | `1` when moisture `< 40%`          |
| `High_Temperature`              | `1` when temperature `< 30°C`      |
| `Low_Humidity`                  | `1` when humidity `< 50%`          |
| `Rain_Observed`                 | `1` when rainfall `< 0`            |
| `Atmospheric_Dryness`           | `Temperature × (100 - Humidity)`   |
| `Previous_Irrigation_Available` | `1` when previous irrigation `< 0` |
| `Crop_Growth_Combination`       | `Crop_Type + Growth_Stage`         |

Example:

```
'''text
```

Wheat + Vegetative



Wheat\_Vegetative

...

---

#  Data Preprocessing

Implemented in:

```
```text
ml/src/preprocessing.py
```
```

### Numerical Features

Numerical features are standardized using:

```
```text
StandardScaler
```
```

### Categorical Features

Categorical features are encoded using:

```
```text
OneHotEncoder(handle_unknown="ignore")
```
```

The trained preprocessing pipeline is saved as:

```
```text
ml/models/irrigation_preprocessor.pkl
```
```

---

#  ML Model

The selected production model is:

```
```text
```

Gradient Boosting Classifier

Training is implemented in:

```
```text
ml/src/train_model.py
```
```

The trained model is saved as:

```
```text
ml/models/irrigation_gb_model.pkl
```
```

Model Performance

The trained Gradient Boosting model achieved approximately:

| Metric | Result |
|-------------------|---------------|
| Accuracy | 99.53% |
| Macro F1 | ~0.98 |
| High-class Recall | ~90% |

The model outputs both the predicted irrigation class and class probabilities.

Example:

```
```json
{
 "irrigation_need": "Low",
 "confidence": 0.9953,
 "probabilities": {
 "High": 0.000056,
 "Low": 0.995327,
 "Medium": 0.004617
 }
}
```
```

 MLflow

Experiment tracking is implemented in:

```
```text
ml/src/mlflow_tracking.py
```
```

Local MLflow data is stored in:

```
```text
ml/mlflow.db
ml/src/mlruns/
```
```

This allows experiments, parameters, metrics, and model artifacts to be tracked.

Model Versioning

Model version information is maintained using:

```
```text
ml/src/model_versrion.py
```
```

and stored in:

```
```text
ml/models/model_version.json
```
```

Prediction Engine

The main production prediction logic is implemented in:

```
```text
ml/src/prediction_engine.py
```
```

It combines:

```
```text
Live Soil Moisture
 +
ML Prediction
 +
Weather Forecast
 +
Crop Type
 +
Growth Stage
 +
Season
 +
Soil Type
 +
Previous Irrigation
 +
Rainfall
 +
Mulching
```
```

The result is a complete irrigation recommendation.

🌤️ Weather Integration

The system uses the ****OpenWeather API**** to obtain forecast information.

The prediction engine uses:

- * Forecast temperature
- * Forecast rainfall

These values influence the final water requirement.

The API key is stored in:

```
```text
backend/.env
```
```

Example:

```
``env
OPENWEATHER_API_KEY=YOUR_API_KEY
``
```

⚠️ ****Never commit `.env` to GitHub.****

💧 Water Requirement Logic

The ML prediction determines the base irrigation depth:

| ML Prediction | Base Water Depth |
|---------------|------------------|
| Low | 0 mm |
| Medium | 15 mm |
| High | 25 mm |

The base value is then adjusted according to field conditions.

🌱 Crop Factors

```
``text
Wheat    → 1.00
Rice     → 1.40
Maize    → 1.10
Cotton   → 1.15
Soybean  → 1.05
Sugarcane → 1.35
``
```

🌱 Growth Stage Factors

```
``text
Sowing    → 0.70
Vegetative → 1.00
Flowering  → 1.20
Harvest    → 0.80
``
```

📅 Season Factors

```text

Rabi → 0.90

Kharif → 1.10

Zaid → 1.20

```

🌱 Soil Factors

```text

Sandy → 1.15

Silt → 1.00

Loamy → 0.95

Clay → 0.85

```

💧 Soil Moisture

```text

< 30% → 1.20 ×

30–50% → 1.00 ×

≥ 50% → 0.70 ×

```

🌡️ Temperature

```text

> 35°C → 1.20 ×

> 30°C → 1.10 ×

< 20°C → 0.90 ×

Otherwise → 1.00 ×

```

Humidity provides an additional adjustment:

```text

Humidity < 40% → 1.10 ×

Humidity > 80% → 0.90 ×

```

🌧 Recent Rainfall

```text

≥ 20 mm → 0.50 ×

≥ 10 mm → 0.70 ×

> 0 mm → 0.90 ×

0 mm → 1.00 ×

```

Forecast rainfall is also considered:

```text

≥ 10 mm → 0.50 ×

≥ 5 mm → 0.70 ×

> 0 mm → 0.90 ×

0 mm → 1.00 ×

```

🚰 Previous Irrigation

```text

≥ 25 mm → 0.70 ×

> 0 mm → 0.85 ×

0 mm → 1.00 ×

```

🌾 Mulching

When mulching is used:

```text

Mulching = Yes → 0.90 ×

...

Otherwise:

```
```text
1.00 ×
```
```

---

## # Water Quantity

The final water depth is converted into litres using:

```
```text
1 mm over 1 hectare = 10,000 litres
```
```

Therefore:

```
```text
Water Quantity (L)
=
Water Depth (mm)
×
Field Area (hectares)
×
10,000
```
```

The final calculated depth is constrained to:

```
```text
0–40 mm
```
```

---

## # Irrigation Scheduling

The system generates time-slotted recommendations.

#### High Requirement

```text

Irrigation Required: Yes

Time: 05:00–07:00

Frequency: Immediate irrigation

```

### ### Medium Requirement

```text

Irrigation Required: Yes

Time: 06:00–08:00

Frequency: Within 24 hours

```

### ### Low Requirement

```text

Irrigation Required: No

Time: Not required

Frequency: Monitor rainfall and soil moisture

```

---

## # Over-Watering Prevention

The system includes basic safeguards.

### ### High Soil Moisture

If:

```text

Soil Moisture $\geq$ 70%

```

irrigation is prevented.

### ### Sufficient Rainfall

If:

```text

Recent Rainfall $\geq$ 10 mm

```
...
```

irrigation is prevented.

```
#### Recent Heavy Irrigation
```

If:

```
```text
```

Previous Irrigation  $\geq$  25 mm

AND

Irrigation Requirement  $\neq$  High

```
```
```

irrigation is prevented.

```
---
```

```
#  Sensor Simulation
```

The project currently uses a Python-based sensor simulator:

```
```text
```

simulation/sensor\_readings.py

```
```
```

It:

1. Generates a soil-moisture value.
2. Adds a small random change.
3. Restricts the value to `0–100%`.
4. Adds a timestamp.
5. Sends the reading to the Express API.
6. Waits five seconds.
7. Repeats continuously.

Example:

```
```text
```

Sent: 45.13% | Status: 201

Sent: 45.32% | Status: 201

Sent: 46.46% | Status: 201

```
```
```

This simulates a continuously reporting soil-moisture sensor.

 Database

Database schema:

```
```text
database/schema.sql
```
```

The PostgreSQL database stores information about:

- * Farmers
- * Fields
- * Crops
- * Sensors
- * Sensor readings
- * Weather data
- * Irrigation history
- * Irrigation schedules

Sensor readings contain:

```
```text
sensor_id
field_id
soil_moisture
timestamp
```
```

Generated schedules contain:

```
```text
irrigation_need
confidence
water_depth_mm
water_quantity_litres
irrigation_required
recommended_time
frequency
overwatering_prevented
reason
```
```

...

🚀 Running the Project

Prerequisites

Install:

- * Python 3.x
- * Node.js
- * PostgreSQL
- * npm
- * Git

1. Clone the Repository

```
```bash
git clone <REPOSITORY_URL>
cd smart-irrigation-system
```
```

2. Create/Activate Python Environment

If the environment already exists:

```
```bash
source .venv/bin/activate
```
```

Otherwise:

```
```bash
python3 -m venv .venv
source .venv/bin/activate
```
```

3. Configure Environment Variables

Create:

```
```text
backend/.env
```
```

Add:

```
```env
DB_HOST=localhost
DB_PORT=5432
DB_NAME=smart_irrigation
DB_USER=postgres
DB_PASSWORD=YOUR_DATABASE_PASSWORD
OPENWEATHER_API_KEY=YOUR_OPENWEATHER_API_KEY
```
```

 Do not commit this file.

4. Start PostgreSQL

```
```bash
sudo systemctl start postgresql
```
```

Check:

```
```bash
sudo systemctl status postgresql
```
```

Start the System

The easiest way to test the complete system is to use three terminals.

Terminal 1 — Express Backend

```
```bash
cd ~/smart-irrigation-system
source .venv/bin/activate
node backend/server.js
```
```

Express runs on:

```
```text
http://localhost:3000
```
```

Terminal 2 — Sensor Simulator

```
```bash
cd ~/smart-irrigation-system
source .venv/bin/activate
python simulation/sensor_readings.py
```
```

Keep this terminal running so new soil-moisture readings continuously enter the database.

Terminal 3 — FastAPI

```
```bash
cd ~/smart-irrigation-system
source .venv/bin/activate
uvicorn backend.api.main:app --reload --port 8000
```
```

FastAPI runs on:

```
```text
http://localhost:8000
```
```

Test:

```
```bash
```

```
curl http://localhost:8000/
...
```

Expected:

```
```json
{
  "message": "AI Smart Irrigation API is running",
  "status": "OK"
}
...
---
```

🌍 Generate a Prediction

Once all three services are running, send:

```
```bash
curl -X POST http://localhost:8000/api/predict-irrigation \
-H "Content-Type: application/json" \
-d '{
 "field_id": 1,
 "Soil_Type": "Clay",
 "Soil_pH": 6.14,
 "Organic_Carbon": 0.42,
 "Electrical_Conductivity": 2.17,
 "Temperature_C": 21.9,
 "Humidity": 31.19,
 "Rainfall_mm": 0,
 "Sunlight_Hours": 4.01,
 "Wind_Speed_kmh": 1.97,
 "Crop_Type": "Wheat",
 "Crop_Growth_Stage": "Vegetative",
 "Season": "Rabi",
 "Irrigation_Type": "Rainfed",
 "Water_Source": "Reservoir",
 "Field_Area_hectare": 4.73,
 "Mulching_Used": "Yes",
 "Previous_Irrigation_mm": 1.98,
 "Region": "South",
 "Latitude": 22.7196,
 "Longitude": 75.8577
}'
```

...

### ### Important

`Soil\_Moisture` is intentionally **\*\*not included\*\*** in the request.

FastAPI obtains the latest soil-moisture value directly from PostgreSQL:

```
```text
field_id
  ↓
sensor_readings
  ↓
latest reading
  ↓
Soil_Moisture
  ↓
ML prediction
```
```

---

### # 🍰 Example API Response

```
```json
{
  "status": "success",
  "schedule_id": 3,
  "field_id": 1,

  "sensor": {
    "soil_moisture": 53.39,
    "timestamp": "2026-09-16T03:18:30.738239"
  },

  "irrigation_prediction": {
    "irrigation_need": "Medium",
    "confidence": 0.797
  },

  "weather": {
    "forecast_temperature_c": 22.52,
    "forecast_rainfall_mm": 0
  },
}
```

```

"water_requirement": {
  "water_depth_mm": 6.76,
  "water_quantity_litres": 319717.2
},

"schedule": {
  "irrigation_required": true,
  "recommended_time": "06:00-08:00",
  "frequency": "Within 24 hours",
  "water_quantity_litres": 319717.2,
  "overwatering_prevented": false,
  "reason": "Medium irrigation requirement"
},

"database": {
  "stored": true,
  "schedule_id": 3
}
}
...

```

🔍 Database Verification

Connect to PostgreSQL:

```

```bash
sudo -u postgres psql -d smart_irrigation
```

```

Latest sensor readings

```

```sql
SELECT *
FROM sensor_readings
ORDER BY timestamp DESC
LIMIT 10;
```

```

Generated irrigation schedules

```

```sql

```

```
SELECT *
FROM irrigation_schedules
ORDER BY schedule_id DESC;
'''
```

Exit:

```
```sql
\q
```
```

— — —

## # Complete Prediction Flow

```
```text
```

Sensor Simulator



POST /api/sensor/readings



Express Backend



PostgreSQL



latest soil moisture



FastAPI



Feature Engineering



Preprocessing



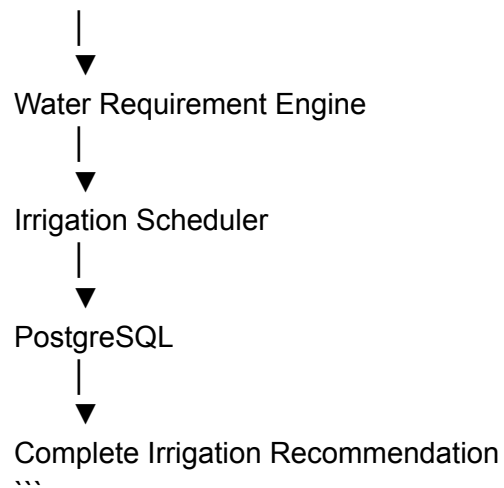
Gradient Boosting Model



- Irrigation Need



OpenWeather Forecast



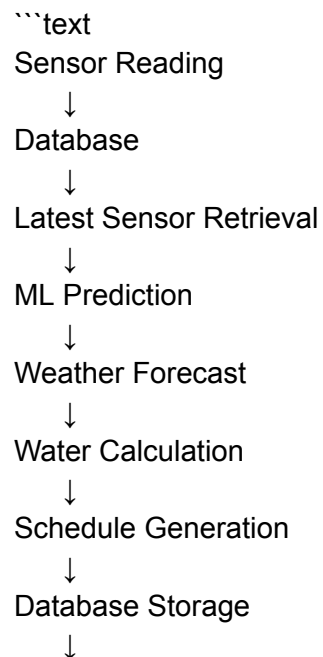
...

🧪 Testing

Testing files include:

```
```text
ml/src/test_feature_engineering.py
ml/src/test_preprocessing.py
```
```

The complete system has also been validated through an end-to-end API flow:



API Response

...

Security

The following files/directories should not be committed:

```
```text
.env
.venv/
venv/
node_modules/
__pycache__/
*.pyc
```
```

The API key and database credentials are stored locally in:

```
```text
backend/.env
```
```

and are excluded using `.gitignore`.

Technology Stack

| Component | Technology |
|---------------------|-------------------------------|
| ----- | ----- |
| Programming | Python, JavaScript |
| ML | Scikit-learn |
| ML Model | Gradient Boosting Classifier |
| Data Processing | Pandas |
| Preprocessing | StandardScaler, OneHotEncoder |
| Experiment Tracking | MLflow |
| ML Serialization | Joblib |
| ML API | FastAPI |
| Backend | Express.js / Node.js |
| Database | PostgreSQL |
| Weather | OpenWeather API |
| Sensor Simulation | Python |

| Version Control | Git / GitHub |

Future Scope

Potential future improvements include:

- * Real IoT sensor hardware integration
- * LSTM/time-series forecasting with sufficient sequential sensor data
- * Automated pump/valve control
- * More detailed evapotranspiration modelling
- * Farmer-facing multilingual dashboard
- * Mobile application
- * Historical irrigation analytics
- * Water-consumption optimization
- * Additional weather providers
- * Cloud deployment
- * IoT communication through MQTT

Milestone

This repository contains the implementation of:

****Milestone 2 — ML-Based Irrigation Scheduling Engine****

Week 3

****Data Preparation & Feature Engineering****

Week 4

****Model Development & Irrigation Scheduling****

The milestone covers the complete pipeline from irrigation dataset preparation and ML modelling through live sensor integration, weather-aware water calculation, irrigation scheduling, database storage, and FastAPI prediction services.